

# My App

Complete Developer Guide

Next.js 14 · TypeScript · Tailwind · Supabase · Prisma · TRPC

Generated: 2026-03-23

## Table of Contents

1. Project Overview
2. Initial Setup
3. CLAUDE.md — The AI Instruction File
4. .claude/settings.json — Hooks & Permissions
5. .claude/commands/ — Slash Commands
6. skills/ — Reusable Skill Definitions
7. agentx/ — Autonomous Agent Definitions
8. plugins/ — Claude Code Plugin System
9. .mcp.json — MCP Server Integration
10. src/ — Application Source Code
11. tests/ — Test Suite
12. docs/ — Documentation
13. scripts/ — Shell Scripts
14. Root Config Files
15. Day-to-Day Developer Workflow

# 1. Project Overview

This project is a Next.js 14 full-stack application with a structured set of AI-assisted development tooling layered on top. There are two distinct layers:

| Layer             | Purpose                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------|
| AI tooling layer  | .claude/, skills/, agentx/, plugins/, .mcp.json — tells Claude Code how to behave in this repo |
| Application layer | src/, tests/, docs/, scripts/, config files — the actual app code                              |

Understanding this separation is key. The AI tooling layer makes Claude Code project-aware, consistent, and automated. The application layer is what gets shipped.

## 2. Initial Setup

### *First-time setup (new developer)*

```
# 1. Clone and enter the repo
git clone https://github.com/HeySchema/Clawd.git
cd Clawd

# 2. Automated bootstrap
bash scripts/setup.sh

# OR manually:
npm install
cp .env.example .env.local # edit with real values
npx prisma generate
npx prisma migrate dev
npm run db:seed
npm run dev
```

### *What .env.local needs*

```
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
DATABASE_URL=postgresql://postgres:password@db.xxxx.supabase.co:5432/postgres
GITHUB_TOKEN=ghp_... # optional, for MCP GitHub
```

Never commit .env.local — it is already in .gitignore.

### 3. CLAUDE.md — The AI Instruction File

#### *What it is*

CLAUDE.md is the single most important file for AI-assisted development. Every time Claude Code starts a session in this repo, it reads this file automatically. It acts as a standing briefing — telling Claude the rules, conventions, and constraints of the project without you having to repeat them.

| Section          | Effect on Claude                                                                   |
|------------------|------------------------------------------------------------------------------------|
| Tech Stack       | Picks the right libraries automatically                                            |
| Conventions      | Writes tests before code, uses conventional commits, never pushes to main          |
| Architecture     | Puts new files in the right directories without being told                         |
| Security         | Refuses console.log(sensitive data), always uses Prisma, validates inputs with Zod |
| Testing          | Knows 80% coverage is required, writes tests for every new function                |
| Git Workflow     | Branches off develop, squash-merges feature branches                               |
| Review Checklist | Self-checks against this list before presenting code                               |

#### *How to modify it*

Edit CLAUDE.md any time your conventions change. Changes take effect in the next session. Keep entries short and imperative.

```
## New Convention (example)
- Always use `dayjs` for dates, never native Date methods
- All new components must have a Storybook story in src/stories/
```

## 4. .claude/settings.json — Hooks & Permissions

### Permissions

```
"permissions": {
  "allow": ["Bash(npm run *)", "Bash(git *)"],
  "deny": ["Bash(rm -rf *)", "Bash(curl * | bash)"]
}
```

allow: Claude can run without prompting. deny: Blocked outright, regardless of context.

### Hooks

| Hook         | When it fires        | Default use                 |
|--------------|----------------------|-----------------------------|
| PreToolUse   | Before any tool call | Safety check script         |
| PostToolUse  | After tool completes | Auto-lint after file writes |
| SessionStart | When session opens   | Echo CLAUDE.md, git fetch   |
| SessionEnd   | When session closes  | Notifications, cleanup      |
| PreCommit    | Before git commit    | lint + typecheck gate       |

### Environment variables

```
"env": {
  "MAX_THINKING_TOKENS": "10000",
  "CLAUDE_AUTOCOMPACT_PCT_OVERRIDE": "50"
}
```

MAX\_THINKING\_TOKENS: Limits reasoning depth. CLAUDE\_AUTOCOMPACT\_PCT\_OVERRIDE: Compacts context at 50% window — preserves more working memory.

### settings.local.json — personal overrides (gitignored)

```
{
  "permissions": { "allow": ["Bash(npm run dev)"] },
  "env": { "DEBUG": "true" }
}
```

## 5. .claude/commands/ — Slash Commands

Slash commands are project-specific shortcuts that expand into full prompts. They live as Markdown files in `.claude/commands/` and are invoked by typing `/command-name`.

| Command                         | What it does                                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>/review [path]</code>     | 7-category review (security, types, tests, perf, a11y, style, conventions). Run before every PR.                    |
| <code>/deploy [env]</code>      | Full pipeline: lint -> typecheck -> tests -> audit -> build -> migrate -> smoke tests. Auto-rollback on failure.    |
| <code>/test-all [flags]</code>  | Runs unit -> integration -> E2E. Fails if coverage < 80%. Supports <code>--coverage</code> , <code>--watch</code> . |
| <code>/bootstrap [flags]</code> | New-developer setup or CI bootstrap. Supports <code>--skip-db</code> , <code>--ci</code> .                          |

### ***Adding your own slash command***

Create any `.md` file in `.claude/commands/` and it is available immediately:

```
# /db-reset — Reset Development Database

## Steps
1. Run `npx prisma migrate reset --force`
2. Run `npm run db:seed`
3. Confirm record counts
```

## 6. skills/ — Reusable Skill Definitions

Skills are reusable, named capability definitions that can be referenced by slash commands, agents, or invoked directly. Unlike one-shot slash commands, skills are designed to be composed.

| Skill          | Purpose                                                                                        |
|----------------|------------------------------------------------------------------------------------------------|
| code-review    | 8-step: security, types, conventions, tests, performance, accessibility, report                |
| security-audit | Deep checklist: auth/RLS, input validation, data exposure, secrets, deps, DB safety            |
| refactor       | 6 safe patterns: Extract Function, Rename, Remove Dead Code, Simplify, Magic Values, Decompose |
| text-writer    | Commit messages, PR descriptions, changelogs, API docs, onboarding guides                      |

### *Adding a new skill*

```
# Skill: Database Migration

## Purpose
Generate and validate Prisma migrations safely.

## Steps
1. Read prisma/schema.prisma
2. Understand the requested change
3. Run: npx prisma migrate dev --name <description>
4. Verify SQL is non-destructive
5. Update src/types/database.ts
```



## 7. agentx/ — Autonomous Agent Definitions

Agents are autonomous workflows that Claude runs end-to-end without step-by-step prompting. Each .yaml file defines a name, tools, trigger, and step sequence.

| Agent                 | Trigger                    | What it does                                                        |
|-----------------------|----------------------------|---------------------------------------------------------------------|
| code-reviewer.yaml    | PR open/update or /review  | Diffs changed files, applies code-review skill, outputs report      |
| test-writer.yaml      | Manual /write-tests        | Finds files below 80%, writes co-located tests, verifies they pass  |
| security-auditor.yaml | Monday 06:00 UTC or manual | npm audit + source scan + security-audit skill -> risk-rated report |
| devops-sre.yaml       | Post-deploy or /deploy     | Full pipeline with auto-rollback if any step fails                  |

### Wire code-reviewer to GitHub Actions

```
# .github/workflows/ai-review.yaml
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: claude-code run agentx/code-reviewer.yaml
```

## 8. plugins/ — Claude Code Plugin System

### *preToolUse — block dangerous actions*

```
async preToolUse({ tool, input }) {
  if (tool === 'Write' && input.file_path?.includes('.env')) {
    return { block: true, reason: 'Direct .env writes not allowed.' };
  }
  return { block: false };
}
```

### *postToolUse — audit logging*

```
async postToolUse({ tool, input, output }) {
  if (tool === 'Write') {
    fs.appendFileSync('audit.log',
      `${new Date().toISOString()} WRITE ${input.file_path}\n`);
  }
}
```

To add a second plugin: create `plugins/my-plugin/index.js` with the same structure and add an entry to `plugins/manifest.json`. Set `enabled: false` to disable without deleting.

## 9. .mcp.json — MCP Server Integration

MCP (Model Context Protocol) servers give Claude direct, authenticated access to external services during a session — no more copy-pasting data into chat.

| Server   | What you can ask Claude                                                                   |
|----------|-------------------------------------------------------------------------------------------|
| GitHub   | "List open bug issues", "Show diff for PR #42", "Create an issue for the auth bug"        |
| Postgres | "Show users table schema", "How many signups this week?", "Find rows where email is null" |

IMPORTANT: Point the Postgres MCP at a read-only replica or dev database — NOT production.

### Adding more servers

```
"slack": {
  "type": "stdio",
  "command": "npx",
  "args": ["-y", "@anthropic/mcp-slack"],
  "env": { "SLACK_TOKEN": "${SLACK_TOKEN}" }
}
```

## 10. src/ — Application Source Code

| Directory       | Contents                             | Conventions                                              |
|-----------------|--------------------------------------|----------------------------------------------------------|
| src/components/ | React UI by feature domain           | PascalCase, named exports, no business logic, accessible |
| src/services/   | Supabase, Prisma, TRPC client        | Only place that talks to DB or auth                      |
| src/utils/      | Pure helpers, Zod validators, logger | No side effects, 100% coverage target                    |
| src/types/      | Shared TS interfaces + DB types      | PascalCase, suffix with Type/Props                       |

### Using the logger (replaces console.log)

```
import { logger } from '@utils/logger';

logger.info('User signed in', { userId: user.id }); // Good
logger.error('Payment failed', { orderId });        // Good
console.log(user);                                 // Bad – caught by PreCommit
```

### Path aliases

```
import { UserType } from '@types'; // -> src/types/index.ts
import { logger } from '@utils/logger'; // -> src/utils/logger.ts
```

## 11. tests/ — Test Suite

| Layer              | Speed | Requires       | Run when           |
|--------------------|-------|----------------|--------------------|
| tests/unit/        | ~1s   | Nothing        | After every change |
| tests/integration/ | ~10s  | Database       | Before pushing     |
| tests/e2e/         | ~60s  | Running server | Before releases    |

### Commands

```
npm run test:unit      # fast, no I/O
npm run test:integration # requires DB
npm run test:e2e       # requires running server
npm test               # all three + coverage
npm run test:watch     # re-runs on file save
npm run test:coverage  # open coverage/index.html
```

### AAA pattern for unit tests

```
describe('slugify', () => {
  it('converts a string to a slug', () => {
    const input = 'Hello World!'; // Arrange
    const result = slugify(input); // Act
    expect(result).toBe('hello-world'); // Assert
  });
});
```

Coverage minimum: 80%. Enforced by vitest --coverage.

## 12. docs/ — Documentation

| File                  | Contents                               | Update when                        |
|-----------------------|----------------------------------------|------------------------------------|
| docs/architecture.md  | System diagram, data flow, auth flow   | Adding major new system components |
| docs/api-reference.md | Every TRPC procedure and REST endpoint | Adding/changing any API procedure  |
| docs/onboarding.md    | Day-1 guide for new developers         | Setup steps or tooling changes     |

## 13. scripts/ — Shell Scripts

| Script     | Usage                                       | What it does                                 |
|------------|---------------------------------------------|----------------------------------------------|
| setup.sh   | bash scripts/setup.sh                       | Full first-time setup. Safe to re-run.       |
| deploy.sh  | bash scripts/deploy.sh [staging production] | Full deploy pipeline. Exits on any failure.  |
| seed-db.sh | bash scripts/seed-db.sh                     | Runs prisma db seed to reset to known state. |

### ***Full reset cycle***

```
npx prisma migrate reset --force && bash scripts/seed-db.sh
```

## 14. Root Config Files

### *tsconfig.json — strict mode*

strict: true and noUncheckedIndexedAccess: true are both enabled. Array access arr[0] returns T | undefined, not T. All nulls must be handled explicitly. No implicit any. Do NOT loosen these settings.

### *Dockerfile — multi-stage build*

| Stage   | Purpose                                      |
|---------|----------------------------------------------|
| deps    | Installs all npm dependencies (cached layer) |
| builder | Generates Prisma client + runs next build    |
| runner  | Minimal production image, non-root user      |

```
docker build -t my-app:latest .  
docker run -p 3000:3000 --env-file .env.local my-app:latest
```

### *.env.example rules*

- Contains every variable name the app needs
- Placeholder values only (your-key-here) — this file IS committed
- Never contains real secrets
- When you add a new env var to the app, add it here first



# 15. Day-to-Day Developer Workflow

## Starting a new feature

```
git checkout develop && git pull
git checkout -b feature/user-profiles
claude
# "Add a user profile page – name, email, avatar.
# Use UserType from src/types, fetch via TRPC."
# Claude will: TRPC procedure -> service -> component -> tests (tests first)
```

## Before opening a PR

```
/review
/test-all --coverage
npm run lint && npm run typecheck
```

## When coverage drops

```
/write-tests
# or: "Run the test-writer agent on src/utils/"
```

## Deploying

```
/deploy staging
/deploy production # after staging smoke tests pass
```

## Quick reference — adding new AI tooling

| What you want                    | What to create                                     |
|----------------------------------|----------------------------------------------------|
| New slash command                | .claude/commands/my-command.md                     |
| New reusable AI capability       | skills/my-skill/SKILL.md                           |
| New autonomous workflow          | agentx/my-agent.yml                                |
| New external service integration | Entry in .mcp.json                                 |
| New safety guard                 | Hook in .claude/settings.json or rule in CLAUDE.md |
| New project-wide AI convention   | Section in CLAUDE.md                               |